

# Week 1 – Linux & Python

INTRODUCTION TO LINUX & PYTHON

ANDY ALUBAIDY, PHD, P.ENG.



**WE ARE**

**HUMBER**

# Agenda for today's training session



## Topic we will be learning today:

- **Course Outline**
- **Critical Path**
- **Introduction to Linux**
- **Introduction to Python**

## Performance Objective:

- **Configure ROS2 on a Linux-based system**
- **Develop ROS2 nodes in Python or C++ using rclpy or rclcpp**

# Syllabus Review-Performance Objectives

- 1. Describe the architecture and core concepts of ROS2 including nodes, publishers/subscribers, services and actions**
- 2. Configure ROS2 on a Linux-based system**
- 3. Develop ROS2 nodes in Python or C++ using rclpy or rclcpp**
- 4. Manage ROS2 workspaces, packages and files by applying structured software development and modular design practices**
- 5. Utilize ROS2 topics, services and actions for inter-node communication**

# Syllabus Review-Performance Objectives

- 6. Integrate hardware components (sensors and actuators) in a ROS2 system. (Tool2)**
- 7. Implement coordinate frame management using the tf2 library to track and transform between reference frames in robotic applications**
- 8. Utilize simulation environments such as Gazebo with ROS2**
- 9. Investigate robot behavior using simulation environment such as Gazebo for testing and debugging**
- 10. Design a complete robotic application using ROS2 to solve open-ended engineering problems**

# Syllabus Review

**This course covers advanced robotics topics that are required for industrial automation such as robot modeling for position and orientation, control and safety circuits, advanced programming, and special features of robots.**

**These concepts include system configurations and applications, power transmission focusing on servo control, types of programming and control, tooling and interfacing with peripherals focusing on I/O control, and feedback devices.**

# Activity - Syllabus Review

**Which of the following are Performance Objectives in this course**

- A. Describe the architecture and core concepts of ROS2 including nodes, publishers/subscribers, services and actions**
- B. Writing bash scripts in Linux for automation**
- C. Develop ROS2 nodes in Python or C++ using rclpy or rclcpp**

# **Syllabus Review-Module's Overview**

**Any changes to the course will be reflected on the critical path document found on BB.**

**Any questions about the Modules or the content to be covered in this course?**

# Syllabus Review-Additional Information

## **Practical Component**

- **Weekly tasks to research and work on prior to next session**
- **Will be due for next session with allotted time for working on the task and assistance during the session**

## **Additional Information**

- **All the classes are going to be held in person.**
- **Attendance: Students missing classes do so at their own risk. They are responsible for all the information discussed in class.**
- **If you are absent for assessments, email your instructor as soon as possible.**

# Course Assessment



- **Lab Assessment – 25%**
  - **Complete lab objective**
- **Midterm Exam – 25%**
- **Final Exam – 25%**
- **Project Report – 25%**

# Class Activity



**Which of the following are not course assessment components?**

- a. Project Report**
- b. Exam**
- c. Project sessions**
- d. Class activities**

Answer: C & D

# Amazon Autonomous Mobile Robot



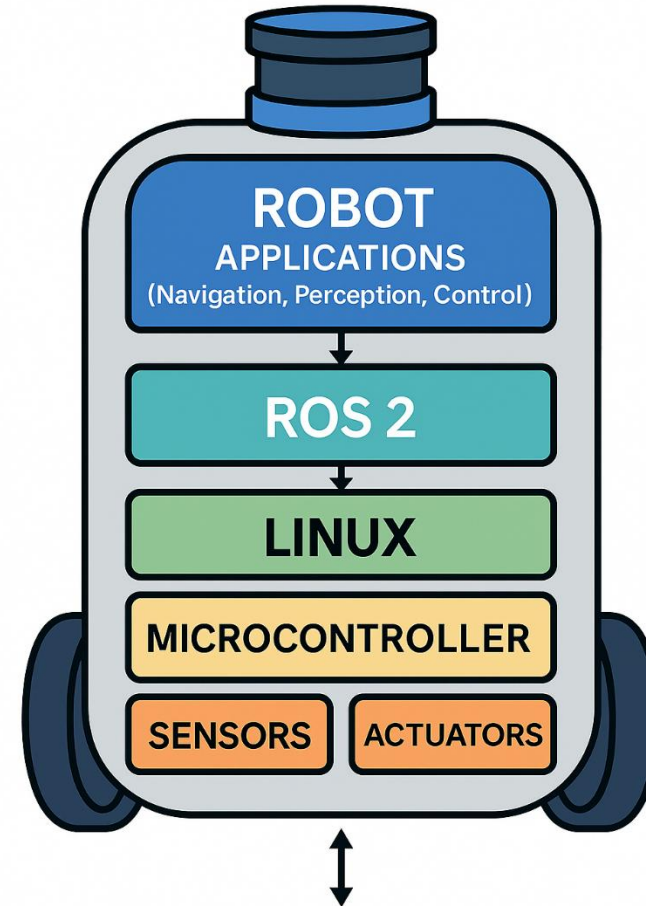
# Waymo Autonomous Driver



# High-Level Architecture



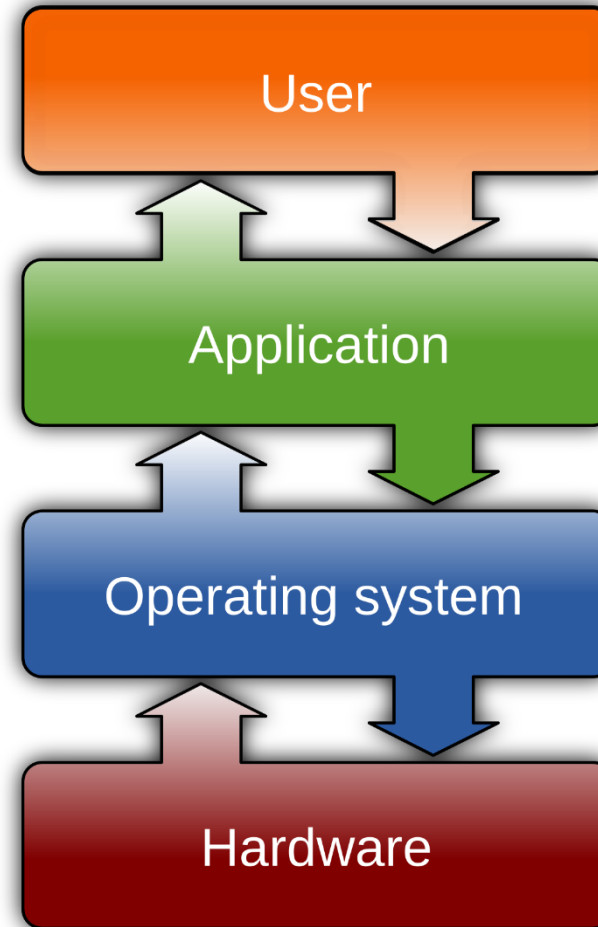
- **Sensors and actuators**
- **Microcontroller**
- **Operating system**
- **ROS 2**
- **Applications**



# Introduction to Operating Systems-What is an OS?



- An operating system (OS) is essential software that manages a computer's hardware and software resources, acting as a bridge between the user and the computer's physical components.
- It facilitates communication between the hardware and applications, enabling users to interact with their devices and perform tasks.



# Introduction to Operating Systems-What is an OS?



Basic Concepts and Differences between Operating Software and Application Software Explained



# What is Linux?



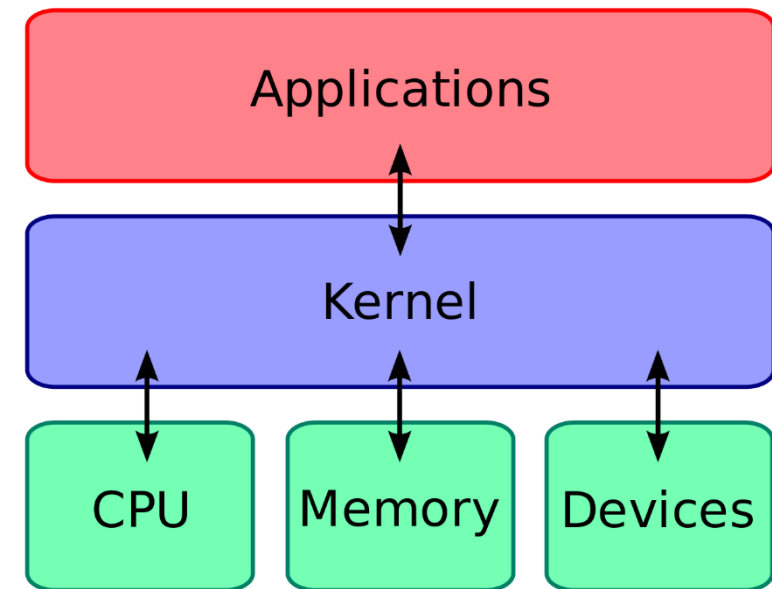
- **Just like Windows, iOS, and Mac OS, Linux is an operating system.**
  - one of the most popular platforms on the planet, Android, is powered by the Linux operating system.
- **Linux has been around since the mid-1990s and has since reached a user-base that spans the globe.**
- **Linux is actually everywhere: It's in your phones, your thermostats, in your cars, refrigerators, Roku devices, and televisions.**

| Beginner-friendly                                                                                                          | Intermediate                                                                                                      | Hard mode                                                                                                                     |
|----------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------|
|                                         |                                |                                            |
|  Ubuntu<br>Based on Debian              |  Garuda Linux<br>Based on Arch |  Arch<br>[Independent] – DIY               |
|  Pop!_OS<br>Based on Ubuntu             |  EndeavourOS<br>Based on Arch  |  Gentoo<br>[Independent] – DIY             |
|  elementary OS<br>Based on Ubuntu (LTS) |  Manjaro<br>Based on Arch      |  Slackware<br>[Independent]                |
|  Mint<br>Based on Ubuntu                |  MX Linux<br>Based on Debian   |  Linux From Scratch<br>[Independent] – DIY |
|  Zorin<br>Based on Ubuntu              |  Fedora<br>Based on Red Hat   |  Qubes OS<br>Based on Fedora – Security   |
|  Solus<br>[Independent]               |  OpenSUSE<br>[Independent]   |  NixOS<br>[Independent] – DIY            |

# What is Linux?



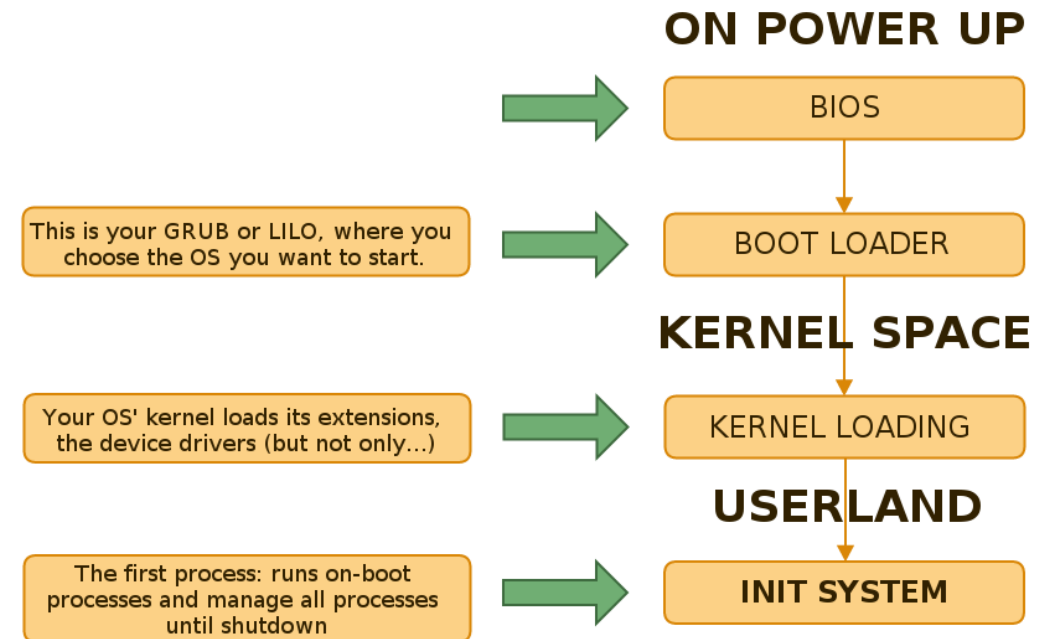
- The Linux operating system comprises several different pieces:
  - **Bootloader** – The software that manages the boot process of your computer. For most users, this will simply be a splash screen that pops up and eventually goes away to boot into the operating system.
  - **Kernel** – This is the one piece of the whole that is called 'Linux'. The kernel is the core of the system and manages the CPU, memory, and peripheral devices. The kernel is the lowest level of the OS.



# What is Linux?



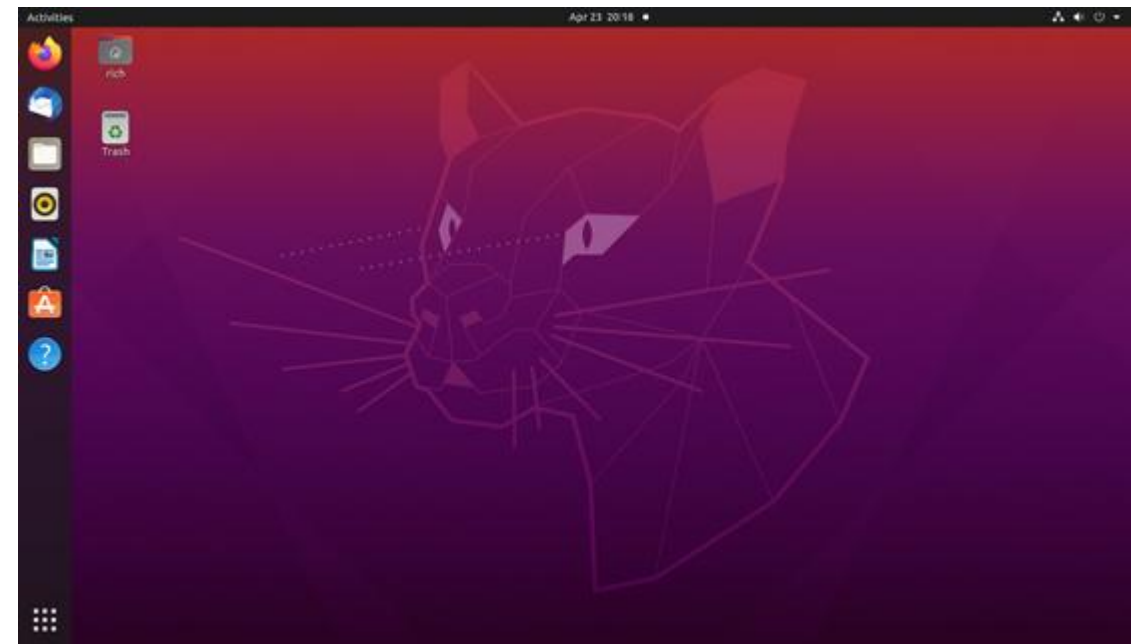
- The Linux operating system comprises several different pieces:
  - **Init system** – This is a sub-system that bootstraps the user space and is charged with controlling daemons. One of the most widely used init systems is systemd, which also happens to be one of the most controversial. It is the init system that manages the boot process, once the initial booting is handed over from the bootloader (i.e., GRUB or GRand Unified Bootloader).
  - **Daemons** – These are background services (printing, sound, scheduling, etc.) that either start up during boot or after you log into the desktop.



# What is Linux?



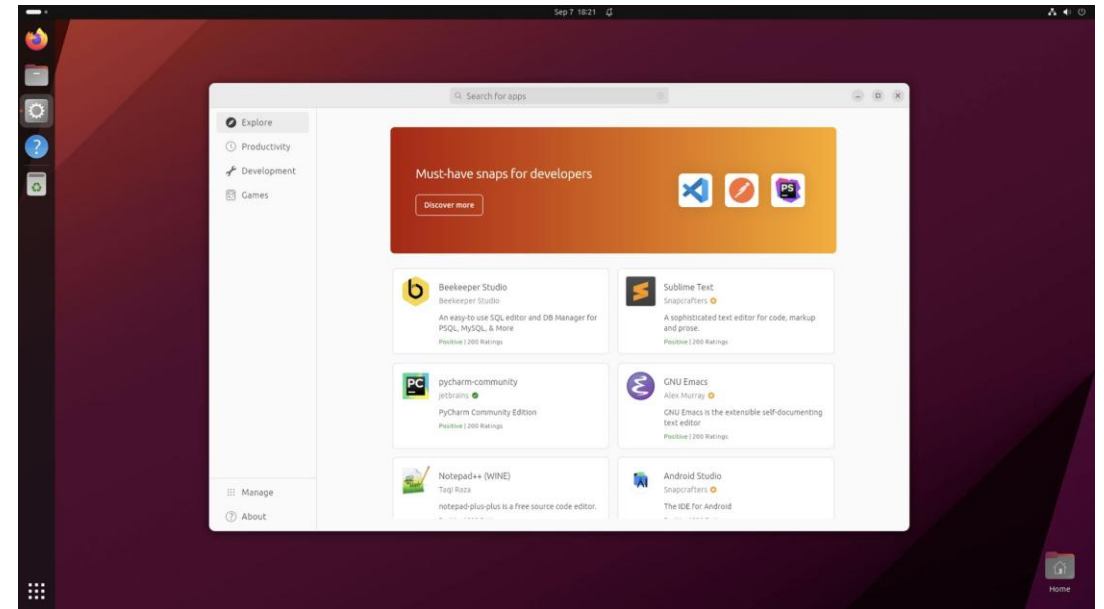
- **The Linux operating system comprises several different pieces:**
  - **Graphical server** – This is the sub-system that displays the graphics on your monitor. It is commonly referred to as the X server or just X.
  - **Desktop environment** – This is the piece that the users actually interact with. There are many desktop environments to choose from (GNOME, Cinnamon, Mate, Pantheon, Enlightenment, KDE, Xfce, etc.). Each desktop environment includes built-in applications (such as file managers, configuration tools, web browsers, and games).



# What is Linux?



- **The Linux operating system comprises several different pieces:**
  - **Applications – Desktop environments do not offer the full array of apps. Just like Windows and macOS, Linux offers thousands upon thousands of high-quality software titles that can be easily found and installed. Most modern Linux distributions (more on this below) include App Store-like tools that centralize and simplify application installation. For example, Ubuntu Linux has the Ubuntu Software Center (a rebrand of GNOME Software) which allows you to quickly search among the thousands of apps and install them from one centralized location.**



# What is Linux?



- The Linux operating system comprises several different pieces:



# Linux Essentials for ROS2

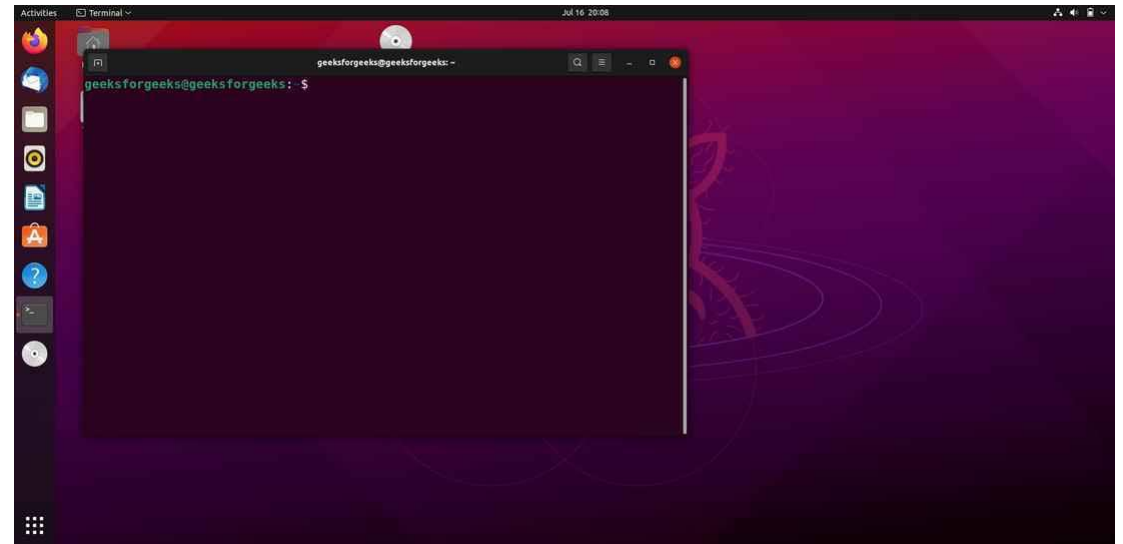


- **Navigating directories with CLI commands**
- **Shell scripting basics**
- **Using package managers (apt)**
- **Permissions and environment variables**



# Linux Terminal

- **The Linux terminal, also known as the shell or command-line interface (CLI), is a text-based application that allows users to interact with the Linux operating system by entering commands. It provides a powerful and efficient way to control the system, manage files, and execute programs, often preferred by experienced users for its speed and flexibility.**



# Linux Essentials



- **Navigating Directories**

| Command | Description             | Example                                      |
|---------|-------------------------|----------------------------------------------|
| cd      | Change directory        | cd ~/ros2_ws/src                             |
| ls      | List contents           | ls -la (shows hidden files and permissions)  |
| pwd     | Print working directory | Shows full path                              |
| mkdir   | Make new directory      | mkdir my_package                             |
| touch   | Create empty file       | touch node.py                                |
| rm      | Remove files            | rm myfile.txt or rm -r mydir for directories |

# Linux Essentials



- **Shell Scripting Basics**
  - **What is a .sh script?**
    - **A text file that contains terminal commands executed in sequence.**
    - **Automate workspace builds**
    - **Launch nodes or simulation with consistent environment setup**

| Syntax                   | Description                     |
|--------------------------|---------------------------------|
| <code>#!/bin/bash</code> | Script interpreter declaration  |
| <code>echo</code>        | Prints a message to the console |
| <code>\$VAR_NAME</code>  | Access variables                |
| <code>chmod +x</code>    | Makes the script executable     |

# Linux Essentials



- **Using package managers (apt)**
  - **Why It Matters**
    - **Installing ROS2 and related dependencies is done through Ubuntu's package manager (apt).**
    - **Knowing how to manage packages is essential.**
    - **sudo gives you root privileges (Administrative Privileges, use with caution).**

| Command                                       | Purpose                         | Example                                            |
|-----------------------------------------------|---------------------------------|----------------------------------------------------|
| <code>sudo apt update</code>                  | Refresh available package lists |                                                    |
| <code>sudo apt install &lt;package&gt;</code> | Install software                | <code>sudo apt install ros-jazzy-desktop</code>    |
| <code>dpkg -i</code>                          | List installed packages         | <code>sudo dpkg -i /tmp/ros2-apt-source.deb</code> |
| <code>apt-cache search &lt;term&gt;</code>    | Search available packages       |                                                    |
| <code>sudo apt remove &lt;package&gt;</code>  | Uninstall a package             | <code>sudo apt remove ros-jazzy-desktop</code>     |

# Linux Essentials



- **Permissions and environment variables**
  - **ROS2 nodes, scripts, and packages need the right permissions to run. Environment variables define how software behaves (e.g., which ROS2 version is active).**

- **Permissions**

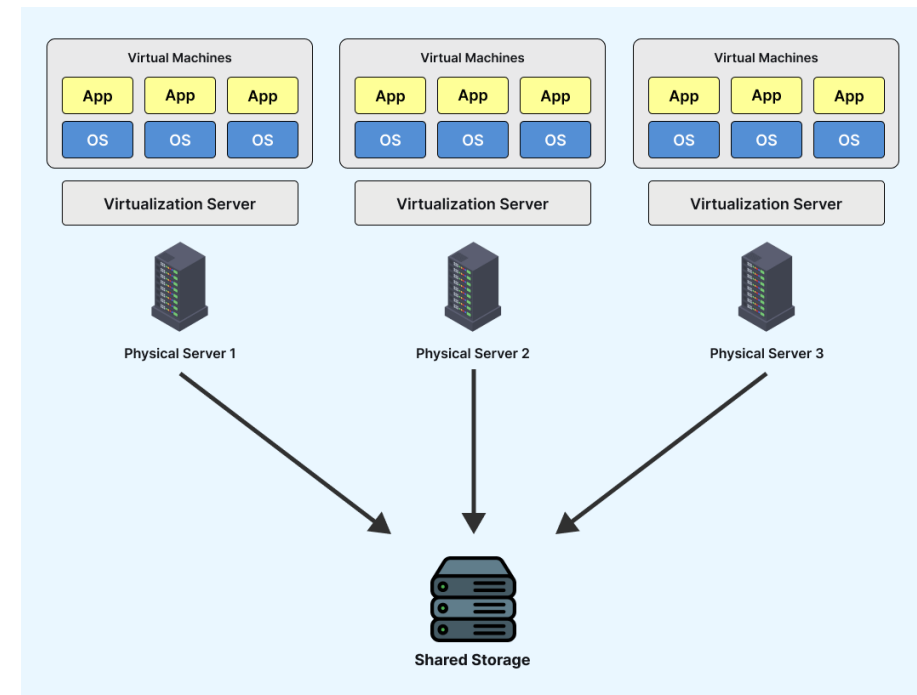
| Command  | Use                    | Example                              |
|----------|------------------------|--------------------------------------|
| chmod +x | Make a file executable | chmod +x launch.sh                   |
| chown    | Change ownership       | sudo chown \$USER:\$USER myscript.sh |

- **Environment Variable**

| Command          | Description                                               | Example                 |
|------------------|-----------------------------------------------------------|-------------------------|
| export VAR=value | Set temporary variable                                    | export ROS_DOMAIN_ID=10 |
| echo \$VAR       | Print variable                                            | echo \$ROS_VERSION      |
| .bashrc          | File where you save environment settings for each session |                         |

# What is a Virtual Machine

- **A virtual machine (VM) is a software-based emulation of a physical computer.**
- **It functions like a separate computer, running its own operating system and applications, but it utilizes the resources of a host computer instead of having its own dedicated hardware.**
- **Think of it as running multiple computers within one physical machine.**



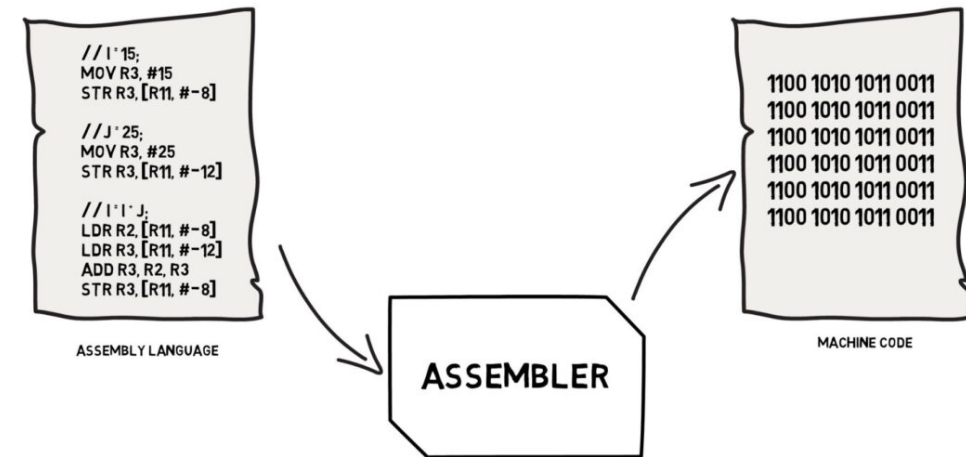
# Break

**15 min break before continuation ...**

# What is a Programming Language



- **Programming languages are the instructions given to a machine to run. Machine instructions are in binary and are called machine code.**
- **Assembly code is an alphanumeric representation of the binary machine code, meant to be more readable for humans**
  - Have set commands ADD, SUB, BEQ, MOV
  - Work with memory registers (physical memory location)
  - Needs an assembler to give machine instructions in machine code
- **A higher-level abstraction language is needed to code in a human readable format for problem solving and easier logical thinking compared to the above low-level languages**



# What is a Programming Language



|           |    |    |    |     |    |     |    |        |    |          |   |        |   |        |
|-----------|----|----|----|-----|----|-----|----|--------|----|----------|---|--------|---|--------|
| 31        | 27 | 26 | 25 | 24  | 20 | 19  | 15 | 14     | 12 | 11       | 7 | 6      | 0 |        |
| funct7    |    |    |    | rs2 |    | rs1 |    | funct3 |    | rd       |   | opcode |   | R-type |
| imm[11:0] |    |    |    |     |    | rs1 |    | funct3 |    | rd       |   | opcode |   | I-type |
| imm[11:5] |    |    |    | rs2 |    | rs1 |    | funct3 |    | imm[4:0] |   | opcode |   | S-type |

## RV64I Base Instruction Set (in addition to RV32I)

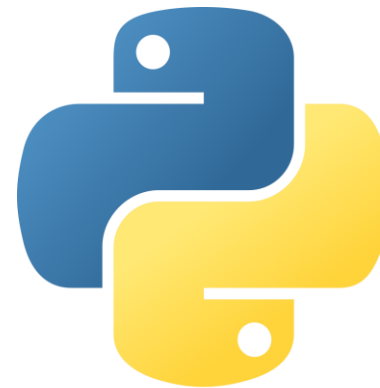
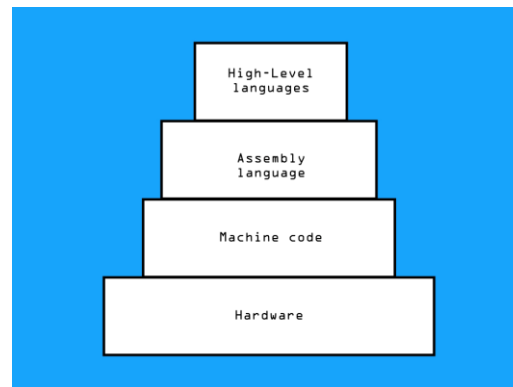
|           |       |     |     |     |          |       |
|-----------|-------|-----|-----|-----|----------|-------|
| imm[11:0] |       | rs1 | 110 | rd  | 0000011  | LWU   |
| imm[11:0] |       | rs1 | 011 | rd  | 0000011  | LD    |
| imm[11:5] |       | rs2 | rs1 | 011 | imm[4:0] | SD    |
| 000000    | shamt | rs1 | 001 | rd  | 0010011  | SLLI  |
| 000000    | shamt | rs1 | 101 | rd  | 0010011  | SRLI  |
| 010000    | shamt | rs1 | 101 | rd  | 0010011  | SRAI  |
| imm[11:0] |       | rs1 | 000 | rd  | 0011011  | ADDIW |
| 0000000   | shamt | rs1 | 001 | rd  | 0011011  | SLLIW |
| 0000000   | shamt | rs1 | 101 | rd  | 0011011  | SRLIW |
| 0100000   | shamt | rs1 | 101 | rd  | 0011011  | SRAIW |
| 0000000   | rs2   | rs1 | 000 | rd  | 0111011  | ADDW  |
| 0100000   | rs2   | rs1 | 000 | rd  | 0111011  | SUBW  |
| 0000000   | rs2   | rs1 | 001 | rd  | 0111011  | SLLW  |
| 0000000   | rs2   | rs1 | 101 | rd  | 0111011  | SRLW  |
| 0100000   | rs2   | rs1 | 101 | rd  | 0111011  | SRAW  |

# What is Python



**Python is a high-level programming language that facilitates the writing of instructions in a human readable format.**

- **Unlike other high level languages python doesn't use a compiler but an interpreter**
- **A compiler converts high level code into machine code instructions for execution**
- **An interpreter converts and executes high level languages line by line**
- **Comes built into the raspberry pi**



# What is Python - Basics



**Python is a statement-based programming language that interprets a line statement one at a time from top of the script to the bottom. The series of statements can work together to define logic and produce results from some input.**

**The statements can have some of the following operations:**

- **Assignment operator : =**
- **Addition operator : +**
- **Subtraction operator : -**
- **Multiplication operator : \***
- **Division operator : /**
- **Exponential operator : \*\***
- **Logical operators: ==, !=, >=, <**
- **Comment: #**
- **print and other built-in functions**

```
total = 19 + 16
# Assign the value of 19 + 16 to the variable total
print(total) # total = 35
# Will show in the console/terminal
```

# What is Python - Functions



Python contains blocks of code called functions that have a certain input parameters and output return associated with the function/method. If a function doesn't have a return, then None type variable is returned.

○ Ex.

```
Sum = add(a, b)
```

```
def function_name(parameter1, parameter2): # Function block definition ends with :
    product = parameter1 * parameter2 # After definition next lines in function block are indented
    sums = parameter1 + parameter2
    difference = parameter1-parameter2
    return product, sums, difference #The output of a function is a returned variable
```

Diagram labels for the function definition:

- Keyword: `def`
- Function Name: `function_name`
- Parameters: `(parameter1, parameter2)`
- Indented Function Body: The block of code inside the function definition, starting with `product = parameter1 * parameter2`.
- Return Statement: `return product, sums, difference`

# What is Python – Function calls



```
# How to call a function
function_results = function_name(1, 3) # call to function with parameter 1 and 2 defined
# function_result = (3, 4, -2)
# if a function has multiple return values it can be put into one variable or individual
product, sums, difference = function_name(5,3)
# product = 15 , sums = 8 , difference = 2
```

# What is Python - Classes



Python is based on object-oriented programming and uses structured blocks called classes that can be instantiated with initial values to create a class object instance. Object classes can have functions associated with the class to be used through the object instance variable.

○ Ex.

```
myRational = Rational(numerator=1, denominator=7)
```

Diagram illustrating the structure of a Python class definition:

```
class Class: # Definition on a class ends with :
    def __init__(self, variable1, variable2, variable3):# intial function call when class object is made
        # Three types of variables
        self.var1 = variable1 # Normal class variable
        self._var2 = variable2 # Protected class variable (marked by _ before name)
        self.__var3 = variable3 # Private class variable (marked by __)
        # initial class function has no return except the class object with these variable values
    def class_function(self, mul_variable): # class functions act the same way as normal functions
        return self.__var3 * mul_variable # return a value and can use private variables
```

Labels and Annotations:

- Keyword:** Points to the `class` keyword.
- Class Name:** Points to the `Class` identifier.
- Indented Class Body:** Points to the entire class definition block.
- Indented Function Body:** Points to the `__init__` and `class_function` methods.

# What is Python – Class Instantiation



```
if __name__ == "__main__": # this line forces the following code to run when the file is run
    rick = Class(1, 2, 3) # when a class is called with its init variables it returns an object
    # The type of variable rick is then an object variable
    product = rick.class_function(16)
    # product = 48 (3 x 16)
    variable1 = rick.var1
    # variable1 = 1 Note*. Private variables cannot be accessed like this
    # Protected variables can be accessed but are not the convention for programmers
```

```
rick = Class(1, variable2=2, variable3=3)
# variables can be explicitly set to avoid confusion
# after first variable is set explicitly all succeeding variables need to be explicit
```

# Class Activity

Fill in the blanks for the following:

- What would the last print statement print

```
class Animal:
    def __init__(self, species, name):
        # species is a private variable dont want to change the type of animal
        self.__species = species
        self.name = name
    def speak(self):
        if self.__species == "Cat":
            print("Meow")
        if self.__species == "Dog":
            print("Bark")

# Tim has a pet dog
Tims_pet = Animal("Dog", "Tommy")
# Tim wants to hear his dog bark
print(Tims_pet.speak())
```

# Class Activity Answers

```
class Animal:
    def __init__(self, species, name):
        # species is a private variable dont want to change the type of animal
        self.__species = species
        self.name = name
    def speak(self):
        if self.__species == "Cat":
            print("Meow")
        if self.__species == "Dog":
            print("Bark")

# Tim has a pet dog
Tims_pet = Animal("Dog", "Tommy")
# Tim wants to hear his dog bark
print(Tims_pet.speak())
```

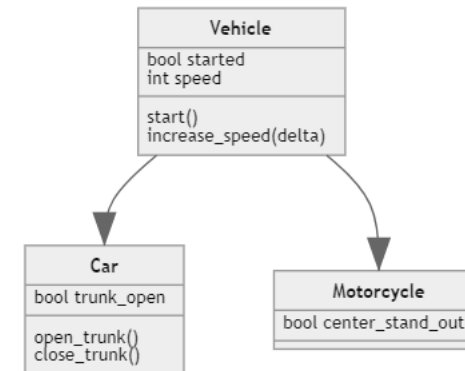
# What is Python - Inheritance



**For certain classes there are specific cases where one general class might not be able to represent the appropriate methods associated with the object instance.**

- All animals don't move the same way (walk, climb, swim, slither) or have the same characteristics
- To allow animal class to remain general a new class would be made for an animal that has unique functions
  - Ex. Snakes slither(), fish swim()
- You wouldn't want to rewrite the entire code from animal class to add that one function or variable
- Inheritance allows you to build a class with the building blocks of another class (variables, methods/functions)

```
class Dog(Animal):  
# The Dog class inherits from the Animal Class
```



# What is Python - Inheritance



Child Class  
Parent Class

```
class Dog(Animal):  
# The Dog class inherits from the Animal Class  
    def __init__(self, species, name, tail_type):  
        super().__init__(species, name) # calls the init function of the parent class  
        # super() also sets up the functions  
        # if instead Animal.__init__() was used instead it would only init the variables  
        self.__tail_type = tail_type # Dogs have specific tail types  
    def action(self): # an action specific to a dog  
        print(f"{self.__Animal__species} Wags {self.__tail_type} tail!") # This is the way to access private parent variables  
        # Can also be accessed by super().species() is species method is setup in parent class to return self.__species
```

Additional  
Function

# What are Data Types



**Variables in python have defined data types for when they are first instantiated, this means only a specific data type can be held by that variable from that point until it is re assigned. The following are the data types that are used in Python:**

- **Int (integer): +/- 1**
  - **Float: 1.2353564**
  - **Long: 15.000000~**
  - **String: "Hello World"**
  - **Character: 'a'**
  - **Boolean: True/False**
  - **List/Array: []**
  - **Dictionary: {'key': value}**
  - **Tuple: ()**
- **Certain types can be converted from one to the other "4" to 4 or 4.12 to 4 using built in functions like int(var), float(var), string(var), etc..**
- **Can use built in function of python called type(var) to find the type of a variable**
- **Data types each have a set amount of fixed memory so when a variable is assigned based on the data type memory is reserved**

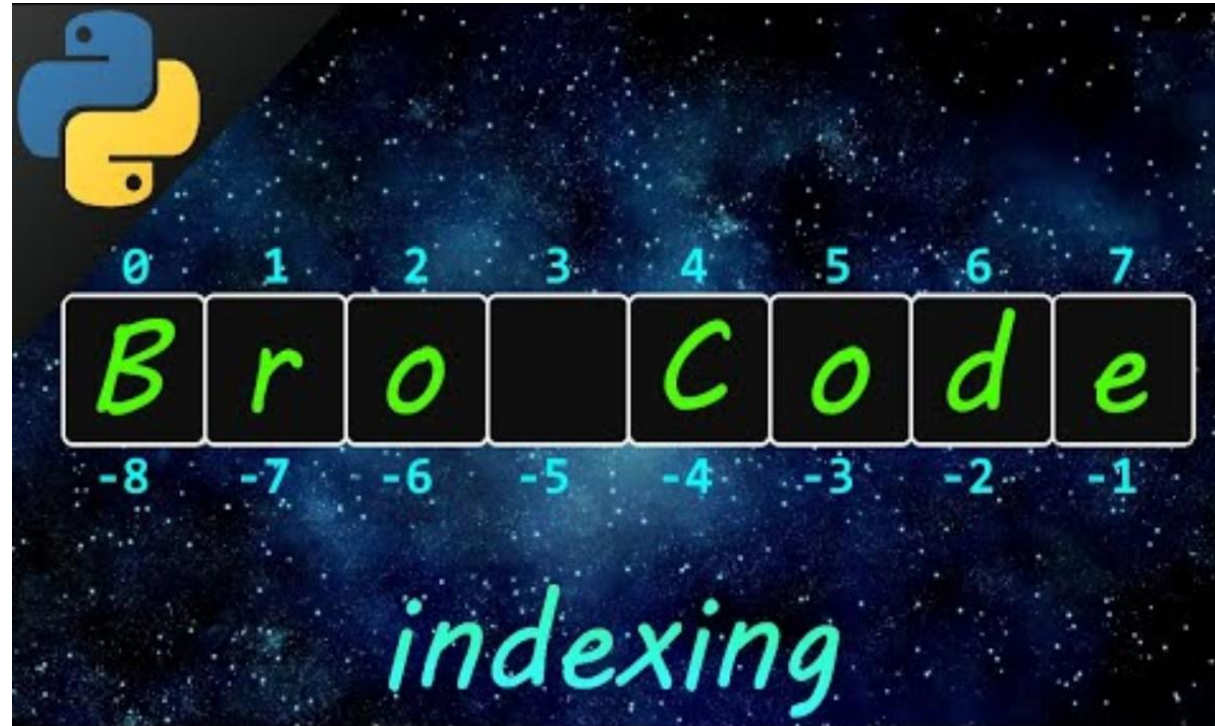
# List and Array Data Types



**List/Arrays are a data type that holds other data within them in an indexed sequence. An index is the specific element in the list/array. The index value starts from 0 and goes till the length of the sequenced array/list. A string is considered as an array of characters and therefore has an index for each character. Elements of arrays can be referenced through the index either using the regular index (positive integers) or the reverse index (negative integers, where -1 is the last item in the sequence).**

|               |     |     |     |    |    |    |    |    |    |    |    |    |  |
|---------------|-----|-----|-----|----|----|----|----|----|----|----|----|----|--|
|               |     |     |     |    |    |    |    |    |    |    |    |    |  |
| Index         | 0   | 1   | 2   | 3  | 4  | 5  | 6  | 7  | 8  | 9  | 10 | 11 |  |
| Character     | H   | e   | l   | l  | o  |    | W  | o  | r  | l  | d  | !  |  |
| Reverse Index | -12 | -11 | -10 | -9 | -8 | -7 | -6 | -5 | -4 | -3 | -2 | -1 |  |
|               |     |     |     |    |    |    |    |    |    |    |    |    |  |

# List and Array Data Types



<https://www.youtube.com/watch?v=ISfxEV4b7po>

# Conditional Logic

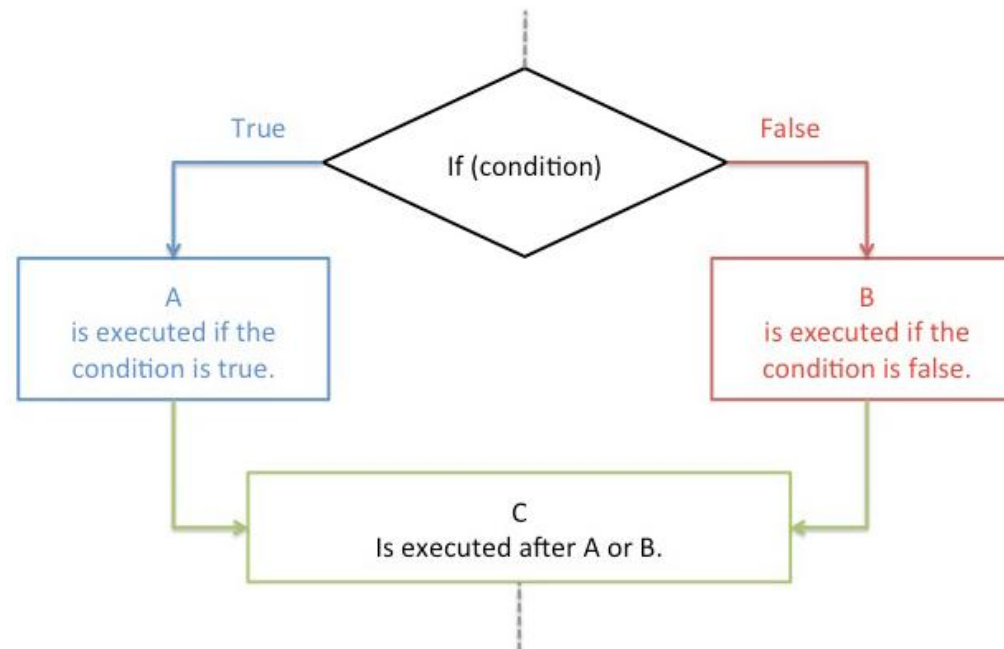


In python to produce any functioning set of code some type of logic is needed to compare results and check for conditions. To following python conditional statements and operators help facilitate the application of such logic:

- If-else statements

- While loops

- For loops



# If – Else Statements



**If – else statements are statements used with the logical operators of python to create a condition that if met would run a specific block of code, and contingently run a different block of code if the statement is not met. Multiple conditions can be stated with elif statements (else-if).**

```
if condition == threshold:
# if condition statement logic states true execute code block
    print('Success')
elif condition == half:
# elif conditions let you check more than one condition
    print('Half Success')
else:
# if all other conditions arent met then this code is executed
    print('Failed')
```

# While Statements



A while statement is used in Python when an incremental or recurrent block of code is needed to be executed. This is done through logical operators to compare the incremental value to see if the condition is met.

- Break statement exits the loop right away and continues the code after the while block
- Continue statement skips to the next iteration of the loop

```
while True:
    if condition == threshold:
        break
    else:
        continue
# Used to run recurrent code till condition is met
```

```
💡 while count < 8:
    if condition == threshold:
        break
    else:
        count = count + 1
        continue # continue is not explicitly needed
# Used to run recurrent code till condition is met
```

# For Statements



**A for statement is for looping to iterate through a set number of iterations, either the number of elements in a list or a fixed number of iterations**

- **Break statement exits the loop right away and continues the code after the block**
- **Continue statement skips to the next iteration of the loop**

```
for item in list: # run through items in list
    item = "checked" # do some work on item
    new_list.append(item)
```

```
for x in range(8): # set range of values for x
    total = total + x
```

# Class Activity



Fill in the blanks to make this block of code logical

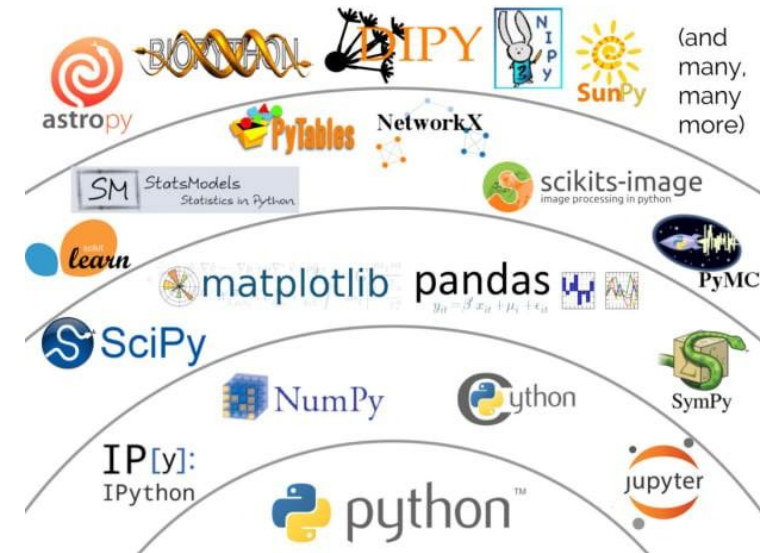
```
# I want to check if my shopping list has candy in it
shopping_list = 
has_candy = False
for  in :
    if item == "Candy":
         True
if has_candy == True:
    print('I have candy in my shopping list')
:
    print("I don't have candy in my shopping list")
```

# What are Libraries



**Libraries are a set of modules that are prewritten code that act to accomplish functions dependent on the library in use. Some common libraries used in python are:**

- **Numpy** – For mathematics
- **Pandas** – For data analysis
- **Matplotlib** – For 2D plotting



# Environment Setup



**It is important to have an environment for the right packages to be installed without interference from older packages**

- **Create a Folder in a location you want to store your packages**
  - `/home/user/environment_name` (example)
- **Run command in terminal : `python -m venv /location/to/environment`**
  - This will create the environment with the name of the folder you created
- **To activate the environment use command:**
  - `source /home/user/environment/bin/activate`
  - This will give you access to the projects packages
  - Use command deactivate to leave environment

```
neatco@raspberrypi:~ $ source /home/neatco/environments/envs/bin/activate
(envs) neatco@raspberrypi:~ $
```

# How to use libraries



To have access to the libraires you would need to use in your own scripts, they would have to be retrieved from the python package index (PyPi). Python upon install also installs Pip which allows you to easily retrieve whatever library you need by name using terminal commands.

- Ex. pip install numpy
- If an environment is active when you use pip command it will install in the environment for use

```
(base) neatco@Neatco-D3:~$ pip install numpy
Collecting numpy
  Downloading numpy-1.26.2-cp39-cp39-manylinux_2_17_x86_64.manylinux2014_x86_64.whl (18.2 MB)
    ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 18.2/18.2 MB 15.2 MB/s eta 0:00:00
Installing collected packages: numpy
ERROR: pip's dependency resolver does not currently take into account all the pa
ckages that are installed. This behaviour is the source of the following depende
ncy conflicts.
daal4py 2021.3.0 requires daal==2021.2.3, which is not installed.
numba 0.54.1 requires numpy<1.21,>=1.17, but you have numpy 1.26.2 which is inco
mpatible.
scipy 1.7.1 requires numpy<1.23.0,>=1.16.5, but you have numpy 1.26.2 which is i
ncompatible.
Successfully installed numpy-1.26.2

[notice] A new release of pip is available: 23.1.2 -> 23.3.1
[notice] To update, run: pip install --upgrade pip
```

# Class Activity

**Get familiar with usage of pip to install packages in the terminal using a virtual environment**

- Find a set of 3 packages to install that are useful

# Key things to remember...



*Read the following articles to better understand Python*

- <https://www.w3schools.com/python/default.asp>
- <https://littlebirdelectronics.com.au/guides/189/introduction-to-python-with-raspberry-pi>
- [Python for Everyone](#) by Dr. Charles Severance

# Summary



- **This week we covered Performance objectives :**
  - **Configure ROS2 on a Linux-based system**
  - **Develop ROS2 nodes in Python or C++ using rclpy or rclcpp**
- **Next week's topics**
  - **What is ROS?**
  - **What is a development environment?**
- **Next week's performance objectives**
  - **Describe the architecture and core concepts of ROS2 including nodes, publishers/subscribers, services and actions.**
  - **Configure ROS2 on a Linux-based system**

THANK YOU.



**WE ARE**

**HUMBER**